

STUDENT MANAGEMENT SYSTEM

Academic Year:- 2025-26

1. PROJECT TITLE

Student Management System

2. PROJECT DESCRIPTION

The Student Management System (SMS) is a C-based console application that helps manage student records efficiently. It allows the user to:

- Add student details
- Display student records
- Search students by roll number
- Update student information
- Delete student records

This project is designed to reduce manual record-keeping and provides a simple computerized solution for handling student data in educational institutions.

3. RESEARCH & BACKGROUND STUDY

Manual handling of student data is time-consuming, prone to errors, and inefficient for large volumes of records. Traditionally, institutions relied on paper-based systems, but with the advent of computers, such management can be automated.

Research showed that:

- A structured database improves accuracy and reduces duplication.
- Simple menu-driven applications in C can be used to practice programming concepts such as structures, arrays, and file handling.
- Student Management Systems are widely used in colleges, coaching classes, and schools.

4. SYSTEM DESIGN & IMPLEMENTATION

Design Approach:

- Input: Roll number, name, age, and marks of students.
- Process: Store data in an array of structures, perform add/search/update/delete operations.
- Output: Display list of students or specific records.

Implementation Details:

- Language Used: C Programming
 - Data Structure: Array of structures
 - Features Implemented:
 1. Add Student
 2. Display Students
 3. Search Student
 4. Update Student
 5. Delete Student
 6. Exit
-

Algorithm for Student Management System

Step 1: Start the Program

Step 2: Initialize

- Define a structure `Student` with the following fields:
 - Roll Number
 - Name
 - Age
 - Marks
- Declare an array `students[MAX]` to store multiple student records.
- Initialize `count = 0` to keep track of the number of students.

Step 3: Display Main Menu

Show the following options repeatedly to the user until they choose to exit:

1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Step 4: Process User Choice

Depending on the option selected by the user, perform the following operations:

Case 1: Add Student

1. Check if `count` has reached `MAX`. If yes, print "Database Full".
2. Otherwise:
 - Input Roll Number, Name, Age, and Marks.
 - Store the details in `students[count]`.
 - Increment `count`.
 - Print "Student added successfully".

Case 2: Display Students

1. If `count == 0`, display "No student records available".
2. Otherwise, loop through all students and print:
 - Roll Number, Name, Age, and Marks.

Case 3: Search Student

1. Ask the user to enter a Roll Number.
2. Search through the `students` array.
3. If found, display the student's details.
4. If not found, print "Student not found".

Case 4: Update Student

1. Ask for the Roll Number of the student to update.
2. Search through the array.
3. If found:
 - Input new Name, Age, and Marks.
 - Update the record.
 - Print "Student details updated".
4. If not found, display "Student not found".

Case 5: Delete Student

1. Ask for the Roll Number of the student to delete.
2. Search through the array.
3. If found:
 - Shift all records after that student one position left.
 - Decrease `count` by 1.
 - Print "Student deleted successfully".
4. If not found, display "Student not found".

Case 6: Exit

- Print "Exiting program".

- Terminate execution.

Step 5: End Program

4.1. Code:-

```
#include <stdio.h>

#include <string.h>

#define MAX 100 // maximum number of students

// structure to store student details
struct Student {
    int roll;
    char name[50];
    int age;
    float marks;
};

// global array of students
struct Student students[MAX];
int count = 0; // total students

// function to add student
void addStudent() {
```

```
    if (count >= MAX) {  
        printf("\nDatabase full! Cannot add more students.\n");  
        return;  
    }  
  
    struct Student s;  
  
    printf("\nEnter Roll Number: ");  
  
    scanf("%d", &s.roll);  
  
    printf("Enter Name: ");  
  
    scanf(" %[^\\n]", s.name);  
  
    printf("Enter Age: ");  
  
    scanf("%d", &s.age);  
  
    printf("Enter Marks: ");  
  
    scanf("%f", &s.marks);  
  
  
    students[count] = s;  
  
    count++;  
  
  
    printf("\nStudent added successfully!\n");  
}  
  
  
// function to display all students  
void displayStudents() {  
    if (count == 0) {  
        printf("\nNo student records available.\n");  
        return;  
    }
```

```
    }

    printf("\n--- Student Records ---\n");
    for (int i = 0; i < count; i++) {
        printf("\nRoll: %d", students[i].roll);
        printf("\nName: %s", students[i].name);
        printf("\nAge: %d", students[i].age);
        printf("\nMarks: %.2f\n", students[i].marks);
        printf("-----\n");
    }
}

// function to search student by roll
void searchStudent() {
    int roll;
    printf("\nEnter Roll Number to search: ");
    scanf("%d", &roll);

    for (int i = 0; i < count; i++) {
        if (students[i].roll == roll) {
            printf("\nStudent Found!\n");
            printf("Roll: %d\nName: %s\nAge: %d\nMarks: %.2f\n",
                students[i].roll, students[i].name, students[i].age, students[i].marks);
            return;
        }
    }
```

Name:-Vedant Ambigar, Roll No:-2541001 , Project Title:- Student Management System

```
    }  
    printf("\nStudent with Roll %d not found!\n", roll);  
}
```

// function to update student details

```
void updateStudent() {  
    int roll;  
    printf("\nEnter Roll Number to update: ");  
    scanf("%d", &roll);  
  
    for (int i = 0; i < count; i++) {  
        if (students[i].roll == roll) {  
            printf("\nEnter New Name: ");  
            scanf(" %[^\\n]", students[i].name);  
            printf("Enter New Age: ");  
            scanf("%d", &students[i].age);  
            printf("Enter New Marks: ");  
            scanf("%f", &students[i].marks);  
  
            printf("\nStudent details updated!\n");  
            return;  
        }  
    }  
}
```

Name:-Vedant Ambigar, Roll No:-2541001 , Project Title:- Student Management System

```
printf("\nStudent with Roll %d not found!\n", roll);  
}
```

// function to delete student

```
void deleteStudent() {  
    int roll;  
    printf("\nEnter Roll Number to delete: ");  
    scanf("%d", &roll);  
  
    for (int i = 0; i < count; i++) {  
        if (students[i].roll == roll) {  
            // shift students  
            for (int j = i; j < count - 1; j++) {  
                students[j] = students[j + 1];  
            }  
            count--;  
            printf("\nStudent deleted successfully!\n");  
            return;  
        }  
    }  
    printf("\nStudent with Roll %d not found!\n", roll);  
}
```

// main function

```
int main() {
```



```
int choice;

while (1) {

    printf("\n===== Student Management System =====");

    printf("\n1. Add Student");

    printf("\n2. Display Students");

    printf("\n3. Search Student");

    printf("\n4. Update Student");

    printf("\n5. Delete Student");

    printf("\n6. Exit");

    printf("\nEnter your choice: ");

    scanf("%d", &choice);


    switch (choice) {

        case 1: addStudent(); break;

        case 2: displayStudents(); break;

        case 3: searchStudent(); break;

        case 4: updateStudent(); break;

        case 5: deleteStudent(); break;

        case 6: printf("\nExiting program...\n"); return 0;

        default: printf("\nInvalid choice! Please try again.\n");

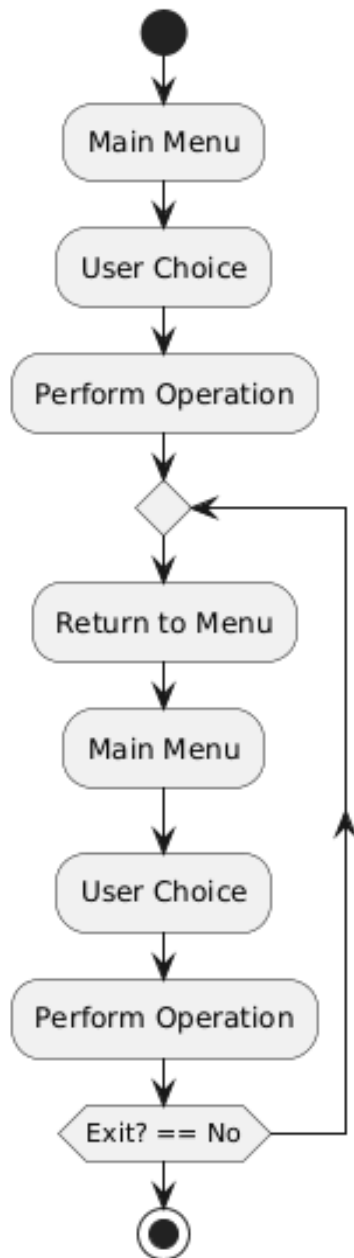
    }

}

return 0;

}
```

4.2.Flow of Execution:



5. TESTING & RESULTS

Test Case	Input	Expected Output	Result
Add Student	Roll=1, Name=Sam, Age=20, Marks=85	Student added successfully	✓
Display Students	-	Shows list with details	✓
Search Student	Roll=1	Displays Sam's details	✓
Update Student	Roll=1, new marks=90	Record updated successfully	✓
Delete Student	Roll=1	Record deleted successfully	✓

5.1. Output:-



```
"E:\vedant c programming\c project\bin\Debug\c project.exe"

===== Student Management System =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice:

Enter Roll Number: 1
Enter Name: sam
Enter Age: 18
Enter Marks: 85

Student added successfully!

===== Student Management System =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 2

--- Student Records ---

Roll: 1
Name: sam
Age: 18
Marks: 85.00

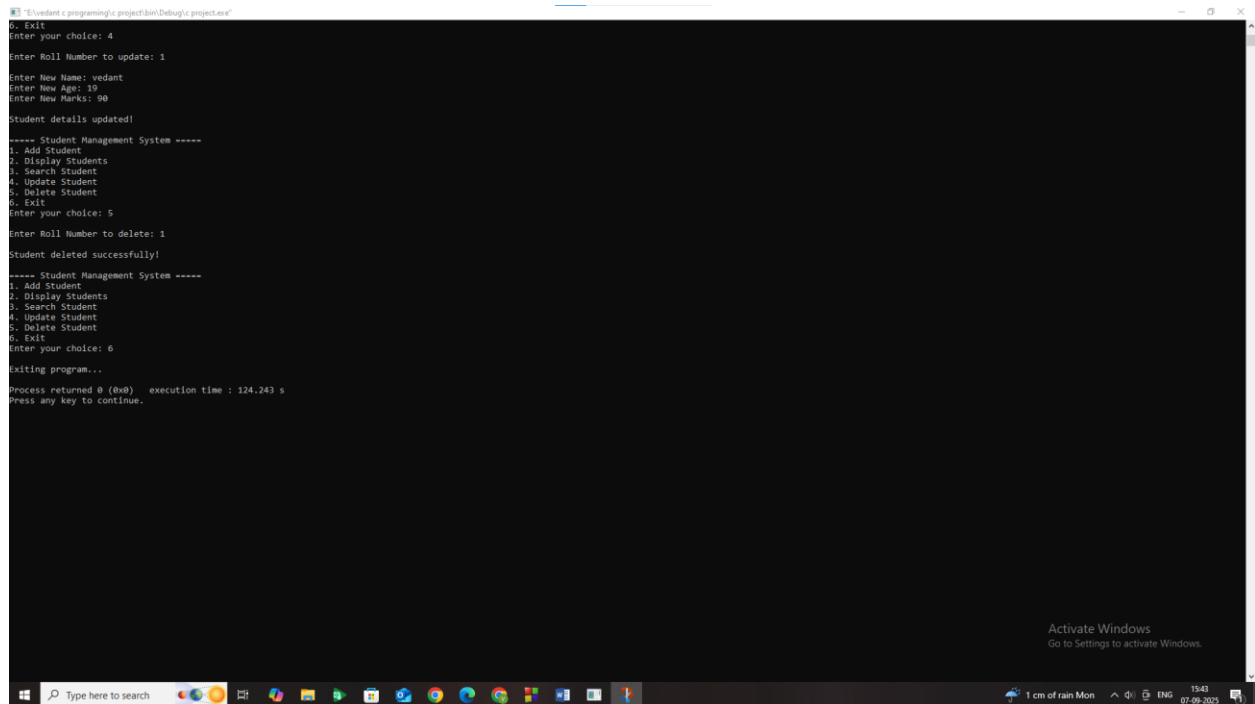
===== Student Management System =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 3

Enter Roll Number to search: 1

Student Found!
Roll: 1
Name: sam
Age: 18
Marks: 85.00

===== Student Management System =====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 4

Enter Roll Number to update: 1
Enter New Name: vedant
```



```
0. Exit
Enter your choice: 4
Enter Roll Number to update: 1
Enter New Name: vedant
Enter New Age: 19
Enter New Marks: 90
Student details updated!

----- Student Management System -----
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 5
Enter Roll Number to delete: 1
Student deleted successfully!

----- Student Management System -----
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 6
Exiting program...
Process returned 0 (0x0)   execution time : 124.243 s
Press any key to continue.
```

6. CONCLUSION & FUTURE ENHANCEMENTS

Conclusion:

The Student Management System project demonstrates how C programming can be applied to solve real-world problems in record management. It improves efficiency, reduces errors, and provides a simple yet effective solution for student data handling.

Future Enhancements:

- Add file handling to store records permanently.
- Implement sorting and filtering (by marks, age, etc.).
- Develop a GUI-based version using advanced programming languages.
- Extend it to a database-connected system (e.g., MySQL).

